

Competitive Programming

Section 12: Stacks, Queues, And Deques

Competitive Programming Course

2026-07-05

Table of contents

1	Why these structures matter	3
2	The three containers	4
3	Monotonic stack	5
4	Why the pop is safe	6
5	Monotonic deque for sliding windows	7
6	Deque over prefix sums	8
7	Worked prefix example	9
8	Common recognition signals	10
9	C++ implementation notes	11
10	Python implementation notes	12
11	Proof habits	13
12	Common wrong approaches	14
12.1	Quadratic nearest-element scan	14
12.2	Forgetting expiration	14
12.3	Keeping values instead of indices	14
12.4	Using two pointers with negative numbers	14
13	Local judge workflow	15
14	Checklist before submitting	16
15	Exercises	17
15.1	A. Next Greater	17
15.2	B. Sliding Window Maximum	17
15.3	C. Shortest Subarray At Least K	17

1 Why these structures matter

Stacks, queues, and deques are not only basic containers. In contests they are often used to keep exactly the candidates that can still matter.

The pattern is:

```
When a new element arrives, remove old candidates made useless by it.
```

This gives linear-time algorithms for problems that look quadratic:

- nearest greater or smaller element;
- sliding window maximum or minimum;
- removing adjacent pairs;
- shortest subarray with a target sum even when values are negative.

2 The three containers

Container	Access pattern	Typical contest use
Stack	last in, first out	nested structure, nearest element
Queue	first in, first out	simulation, BFS later
Deque	push/pop both ends	sliding windows, monotonic candidates

In C++:

```
vector<int> st;      // often best as a stack
queue<int> q;
deque<int> dq;
```

In Python:

```
stack = []
from collections import deque
dq = deque()
```

3 Monotonic stack

A monotonic stack stores candidates in sorted order by value.

For next greater element to the right, scan from right to left and keep a stack of values that are strictly greater candidates.

When processing $a[i]$, any stack top $\leq a[i]$ is useless:

$a[i]$ is closer to the left and at least as large, so that old value can never be the first greater value for any earlier element.

Example:

```
a = 2 1 3 2 5
```

Scan from right:

```
i=4, value=5: stack empty -> ans=-1, push 5
i=3, value=2: top 5 > 2 -> ans=5, push 2
i=2, value=3: pop 2, top 5 > 3 -> ans=5, push 3
i=1, value=1: top 3 > 1 -> ans=3, push 1
i=0, value=2: pop 1, top 3 > 2 -> ans=3, push 2
```

Answer:

```
3 3 5 5 -1
```

4 Why the pop is safe

Suppose we scan from right to left and see current value x .

If the stack top is $y \leq x$, then y is to the right of x , but x is closer to every future position we will process. Since x is at least as large as y , y can never be the first strictly greater value for a future position.

So y can be removed permanently.

That is the core amortized idea:

`each element is pushed once and popped once`

5 Monotonic deque for sliding windows

For window maximums, the front of the deque should always hold the index of the maximum value in the current window.

Maintain two invariants:

1. indices in the deque are inside the current window;
2. values in the deque are decreasing from front to back.

When a new index i arrives:

1. Remove front indices that are too old.
2. Remove back indices whose values are $\leq a[i]$.
3. Push i .
4. Once the first full window exists, the answer is `a[dq.front()]`.

Example:

```
a = 1 3 -1 -3 5 3 6 7
k = 3
```

Window maximums:

```
[1, 3, -1] -> 3
[3, -1, -3] -> 3
[-1, -3, 5] -> 5
[-3, 5, 3] -> 5
[5, 3, 6] -> 6
[3, 6, 7] -> 7
```

6 Deque over prefix sums

With only nonnegative numbers, a two-pointer window can find short subarrays with sum at least K . Negative numbers break that, because extending the right endpoint can make the sum smaller.

Use prefix sums:

```
sum(l, r) = pref[r] - pref[l]
```

For each right endpoint r , we want the largest possible l among candidates with:

```
pref[r] - pref[l] >= K
```

But we also want small prefix sums, because smaller $\text{pref}[l]$ makes the difference larger.

Keep prefix indices in a deque with increasing prefix values.

For current prefix $\text{pref}[i]$:

1. While the front gives a valid sum, update the answer and pop it. Popping is safe because later i values only make the length longer for that same front.
2. While the back prefix is $\geq \text{pref}[i]$, pop it. The current index is later and has a smaller or equal prefix sum, so the old back is dominated.
3. Push i .

7 Worked prefix example

Array:

```
2 -1 2 -1 2 4
K = 5
```

Prefix sums:

```
index: 0  1  2  3  4  5  6
pref:  0  2  1  3  2  4  8
```

At $i = 6$, $\text{pref}[i] = 8$.

The deque contains useful low prefix sums. If the front is index 4 with $\text{pref}[4] = 2$, then:

```
8 - 2 = 6 >= 5
length = 6 - 4 = 2
```

The subarray is:

```
a[4] + a[5] = 2 + 4 = 6
```

So the answer can be 2.

8 Common recognition signals

Use a monotonic stack when the statement asks for:

- next greater;
- next smaller;
- nearest taller building;
- stock span;
- remove dominated candidates to the left or right.

Use a monotonic deque when the statement asks for:

- maximum/minimum over every fixed-size window;
- shortest/longest interval with a prefix-sum condition;
- DP transition over a sliding range.

Use a plain queue when the statement is a literal FIFO process or, later, BFS.

9 C++ implementation notes

For stacks, `vector<T>` is often faster and more flexible than `stack<T>`:

```
vector<long long> st;  
st.push_back(x);  
st.pop_back();  
st.back();
```

For deques:

```
deque<int> dq;  
dq.push_back(i);  
dq.pop_front();  
dq.pop_back();
```

Use indices, not values, when you must know whether an element has left a window.

10 Python implementation notes

Use a list as a stack:

```
stack.append(x)
stack.pop()
stack[-1]
```

Use `collections.deque` for efficient front pops:

```
from collections import deque
dq = deque()
dq.popleft()
```

Avoid `list.pop(0)`; it is linear time.

11 Proof habits

For monotonic structures, prove two things:

1. **Dominance.** When you pop a candidate, explain why it can never be useful again.
2. **Amortization.** Each element is inserted once and removed at most once, so total time is linear after input reading.

The code is short, but the proof is the reason it is trusted.

12 Common wrong approaches

12.1 Quadratic nearest-element scan

Checking all positions to the right for each index is too slow for $n=200000$.

12.2 Forgetting expiration

For sliding windows, a large value can stay in the deque after it leaves the window unless you remove old indices from the front.

12.3 Keeping values instead of indices

For window problems, values alone do not tell you whether the candidate is still inside the window.

12.4 Using two pointers with negative numbers

If numbers can be negative, window sums are not monotone under expansion. Use prefix sums and a deque instead.

13 Local judge workflow

Run Section 12:

```
python3 sections/12_stacks_queues_deques/check.py
```

Run the reference solutions:

```
CP_TARGET=solution python3 sections/12_stacks_queues_deques/check.py
```

The random tests compare against brute force on small arrays. That is useful for this topic because off-by-one expiration and equality choices are common.

14 Checklist before submitting

- Decide whether equality should be popped: $<$, $\leq$, $>$, or $\geq$.
- Store indices when window membership matters.
- Remove expired indices before reading the front.
- Use prefix sums for subarray sums.
- Use 64-bit integers for prefix sums in C++.
- Prove every popped element is dominated.
- Test duplicates, all increasing, all decreasing, and negative values.

15 Exercises

15.1 A. Next Greater

Practice a right-to-left monotonic stack.

15.2 B. Sliding Window Maximum

Practice a decreasing deque of indices.

15.3 C. Shortest Subarray At Least K

Practice a monotonic deque over prefix sums, including negative values.